

Generalization

Lecturer: Jiaming Liang

September 24, 2026

Training a machine learning model means choosing parameters that perform well on the training data. However, our ultimate goal is not to perform well on data that we have already seen. We want the model to perform well on new, unseen data. This distinction leads to two fundamental questions:

- How should we select a model using the available data?
- How well does performance on the training data predict performance on unseen data?

The first question motivates the use of training, validation, and test sets. The second question leads to the concept of *generalization*.

1 Model Selection

Suppose that we have a dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n.$$

A standard approach is to divide the data into three parts:

$$\mathcal{D}_{\text{train}}, \quad \mathcal{D}_{\text{val}}, \quad \mathcal{D}_{\text{test}}.$$

Training set. The training set is used to learn the model parameters. For example, we may solve

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{train}}} L_i(\boldsymbol{\theta}).$$

Validation set. The validation set is used to make choices that are not directly learned by the optimization algorithm. These choices are often called *hyperparameters*.

For example, in ridge regression,

$$\hat{\boldsymbol{\theta}}_{\lambda} = \arg \min_{\boldsymbol{\theta}} \left\{ \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{\text{train}}} L_i(\boldsymbol{\theta}) + \lambda \|\boldsymbol{\theta}\|^2 \right\},$$

the parameter vector $\boldsymbol{\theta}$ is learned from the training data, while the regularization parameter λ is a hyperparameter. We can train models using several candidate values of λ , evaluate them on the validation set, and select the value with the best validation performance. Other examples of hyperparameters include the degree of a polynomial model, the depth of a decision tree, the number of neighbors in k -nearest neighbors, and the architecture of a neural network.

Test set. The test set is used to evaluate the final model after all model-selection decisions have been made. It is important that the test set not be repeatedly used to choose models or hyperparameters. If we repeatedly select models based on their test performance, then information from the test set has effectively entered the training procedure, and the reported test performance may become overly optimistic.

Thus, the basic principle is

training data: fit parameters validation data: select hyperparameters test data: final evaluation.

2 Cross-Validation

A separate validation set is simple and effective when a large amount of data is available. When the dataset is relatively small, however, setting aside a large validation set leaves fewer examples for training.

A common alternative is *K-fold cross-validation*. We divide the available training data into K approximately equal subsets, or *folds*, $\mathcal{D}_1, \dots, \mathcal{D}_K$.

For each $k = 1, \dots, K$,

1. use $\mathcal{D}_k$ as the validation set;
2. train the model using the other $K - 1$ folds;
3. compute the validation error on $\mathcal{D}_k$.

If $E_{\text{val}}^{(k)}$ denotes the validation error in fold k , the cross-validation error is

$$E_{\text{CV}} = \frac{1}{K} \sum_{k=1}^K E_{\text{val}}^{(k)}.$$

We can repeat this procedure for different models or hyperparameter values and select the one with the smallest cross-validation error. Typical choices are $K = 5$ or $K = 10$. Cross-validation allows every training example to be used for both training and validation, although in different folds. The cost is that the model must be trained K times for every hyperparameter choice.

The test set remains separate from the cross-validation procedure and should only be used for final evaluation.

3 Generalization from Finite Data

We now ask a more fundamental question: why should performance on a finite training set tell us anything about performance on unseen data?

Let $Z = (X, Y)$ denote a random example drawn from an unknown data distribution P , and let h be a prediction rule. The *population risk*, or true risk, is

$$R(h) = \mathbb{E}_{Z \sim P} [\ell(h; Z)],$$

where $\ell(h; Z)$ denotes the loss of h on example Z . This is the quantity that we ultimately care about, since it measures performance on new data. However, the distribution P is unknown. Given independent training examples

$$Z_1, \dots, Z_n \stackrel{\text{i.i.d.}}{\sim} P,$$

we can instead compute the *empirical risk*

$$\hat{R}_n(h) = \frac{1}{n} \sum_{i=1}^n \ell(h; Z_i).$$

The difference

$$R(h) - \hat{R}_n(h)$$

is called the *generalization gap*. The key question is:

How close is the empirical risk to the population risk?

3.1 Concentration for a Fixed Model

Suppose that the loss is bounded:

$$0 \leq \ell(h; Z) \leq 1.$$

For a fixed hypothesis h , the quantities

$$\ell(h; Z_1), \dots, \ell(h; Z_n)$$

are independent random variables taking values in $[0, 1]$. Their average is $\hat{R}_n(h)$, while their expectation is $R(h)$.

Hoeffding's inequality gives

$$\mathbb{P} \left(\left| \hat{R}_n(h) - R(h) \right| \geq \varepsilon \right) \leq 2 \exp(-2n\varepsilon^2).$$

Thus, for a fixed model, the empirical risk becomes increasingly concentrated around the population risk as the sample size n increases. For example, the probability that the generalization gap exceeds a fixed $\varepsilon > 0$ decreases exponentially with n . There is, however, an important difficulty. In machine learning, the model h is usually *not* chosen before observing the data. Instead, we use the data to choose one model from a collection of candidate models.

3.2 A Finite Hypothesis Class

Suppose that we choose a model from a finite hypothesis class

$$\mathcal{H} = \{h_1, \dots, h_M\}, \quad |\mathcal{H}| = M.$$

For each fixed $h \in \mathcal{H}$, Hoeffding's inequality gives

$$\mathbb{P} \left(\left| \widehat{R}_n(h) - R(h) \right| \geq \varepsilon \right) \leq 2e^{-2n\varepsilon^2}.$$

We would like this guarantee to hold *simultaneously for every* $h \in \mathcal{H}$. Recall the *union bound*: for any events $A_1, \dots, A_M$,

$$\mathbb{P} \left(\bigcup_{j=1}^M A_j \right) \leq \sum_{j=1}^M \mathbb{P}(A_j).$$

Applying the union bound over all hypotheses gives

$$\mathbb{P} \left(\exists h \in \mathcal{H} : \left| \widehat{R}_n(h) - R(h) \right| \geq \varepsilon \right) \leq \sum_{h \in \mathcal{H}} \mathbb{P} \left(\left| \widehat{R}_n(h) - R(h) \right| \geq \varepsilon \right) \leq 2|\mathcal{H}|e^{-2n\varepsilon^2}.$$

Equivalently,

$$\mathbb{P} \left(\sup_{h \in \mathcal{H}} \left| \widehat{R}_n(h) - R(h) \right| \geq \varepsilon \right) \leq 2|\mathcal{H}|e^{-2n\varepsilon^2}.$$

Now choose a confidence level $0 < \delta < 1$ and set

$$2|\mathcal{H}|e^{-2n\varepsilon^2} = \delta.$$

Solving for ε gives

$$\varepsilon = \sqrt{\frac{\log(2|\mathcal{H}|/\delta)}{2n}}.$$

We therefore obtain the following generalization bound: Suppose that $0 \leq \ell(h; Z) \leq 1$ for every $h \in \mathcal{H}$, and suppose that $\mathcal{H}$ is finite. Then, with probability at least $1 - \delta$,

$$\sup_{h \in \mathcal{H}} \left| R(h) - \widehat{R}_n(h) \right| \leq \sqrt{\frac{\log(2|\mathcal{H}|/\delta)}{2n}}.$$

Importantly, this bound holds simultaneously for every $h \in \mathcal{H}$. Therefore, it also holds for a model selected using the training data.

3.3 What Does the Bound Tell Us?

The bound has the general form

$$\text{generalization gap} \lesssim \sqrt{\frac{\text{model complexity} + \log(1/\delta)}{\text{number of training examples}}}.$$

It captures several important ideas. First, increasing the number of training examples improves generalization:

$$n \uparrow \quad \implies \quad \text{generalization bound} \downarrow.$$

Second, choosing among more candidate models makes generalization more difficult:

$$|\mathcal{H}| \uparrow \quad \implies \quad \text{generalization bound} \uparrow.$$

Notice that the dependence on the number of models is only logarithmic:

$$\sqrt{\frac{\log |\mathcal{H}|}{n}}.$$

Finally, demanding a higher-confidence guarantee also makes the bound larger. A smaller δ corresponds to a guarantee that holds with higher probability $1 - \delta$.

The main lesson is that minimizing training error alone is not enough. Generalization depends on both the amount of available data and the complexity of the class of models from which we select.